



PROJECT OVERVIEW

slideotter

A local workbench for structured presentations that stay grounded, reviewable, and easy to publish.

This tour shows new users where the work lives and how an idea becomes an archived PDF.

Structured source

Slides, context, sources, and materials stay inspectable in the project tree.

Live workbench

Preview, edit, attach images, compare variants, and plan decks in one browser flow.

Guarded output

Generated changes wait for review, then validation protects the archive.

TOUR MAP

What you will learn

Choose a deck, ground it, review changes, and ship a checked archive.

Start

Pick or create a deck with constraints and sources.

Author

Edit JSON, attach materials, keep context close.

Review and ship

Compare candidates, run checks, and archive the intended version.

The flow works for the demo or a new deck.

The problem and the loop

PROBLEM

Why decks get messy

Deck work gets hard when source, preview, and final PDF stop agreeing.

— The file no longer explains the visible slide.

— Audience and tone live outside the deck.

WORKFLOW

A simple authoring loop

Most work follows four steps and leaves readable local state.

Daily flow

1

Plan

Capture objective, outline, audience.

2

Edit

Change text or JSON beside preview.

3

Review

Compare alternatives before applying.

Why it stays calm

Local

Runs against this repository.

Explicit

Buttons request bounded actions.

Inspectable

State stays readable on disk.

Author in the workbench

WORKSPACE

Choose a presentation first

slideotter manages multiple local decks with visual first-slide cards.

What a deck owns

Slides

Specs under presentations/<id>/slides.

Context

Brief, outline, theme, checks.

Sources

Notes, excerpts, URLs.

What you can do there



Create

Set brief, length, sources.



Duplicate

Copy a deck when you want a branch.



Delete

Remove a deck only after confirmation.

CONTEXT

Give the deck a brief and sources

Brief first

Before generation, state the goal and trusted material.

- What should change?
- First-draft slide count.

SOURCE MODEL

Slides are readable JSON

Each structured slide is a compact JSON file. The renderer turns those fields into the slide you inspect in the studio.

Current slide families

Cover

Identity, summary, logo, cards.

TOC

Tour framing and section cards.

Content

Evidence and guardrail panels.

Why JSON helps

Parseable

Invalid JSON cannot be saved.

Diffable

Changes stay reviewable in source.

Extensible

New families can be added later.

Edit with the slide in view

The main preview is the working surface. Text and JSON edits update it immediately.

Edit path

1

Select

Choose the target in the slide strip.

2

Inline

Double-click editable preview text.

3

JSON

Open the spec drawer.

What prevents mistakes

Parse errors block save

Invalid JSON cannot persist.

Selected slide only

Actions stay scoped.

Preview stays central

Editing chrome is secondary.

Attach images as materials

Upload, search, or provide starter images. Materials stay with the deck and carry source information.

Material flow

Upload

Files land in presentations/<id>/materials.

Search

Openverse and Wikimedia can import images.

Credit

Provider, creator, license, and source are saved.

Practical rules



Alt text

Make the image understandable.



Source line

Captions keep visible credit close.



Explicit attach

Upload alone never edits a slide.

RUNTIME

One renderer keeps previews honest

The studio avoids separate preview and export logic. The same DOM slide powers authoring, comparison, validation, and PDF output.

— The large slide in Slide Studio.

— Selector and presentation cards.

Review before applying

VARIANTS

Generate options, not surprises

Review options

Generation creates review candidates, reports status, and keeps the current slide unchanged until Apply.

- Try wording, structure, layout, or theme.
- Use local rules, OpenAI, LM Studio, or OpenRouter.

REVIEW

Compare before applying

When candidates exist, the compare workspace becomes the decision surface: current slide, candidate slide, changed fields, and apply buttons.

What to inspect

Current

The slide you would replace.

Candidate

The proposed version.

Changes

Touched fields and line diffs.

Decision cues

Hidden until useful

No empty comparison area.

Direct selection

Rows choose the candidate to inspect.

No surprise writes

Nothing changes without Apply.

OUTLINE DRAWER

Plan the deck from Slide Studio

The Outline drawer keeps sources, reusable plans, length scaling, and deck-context proposals beside the active slide.

Planning modes

1

Brief

Edit shared deck context.

2

Plans

Generate reusable outlines.

3

Changes

Propose deck updates.

Review rules

Skip, do not delete

Skipped slides can return.

Paths are named

Review touched files.

Apply once

Plan promotion is explicit.

ASSISTANT

Use chat for bounded tasks

In Slide Studio, chat launches bounded workflows and keeps history in a Log tab.

Inputs it sees

Message

The request you type.

Selection

Text from the preview.

Slide

Current slide id and context.

How it stays inspectable

Known actions

Triggers studio workflows.

Studio only

Hidden while browsing decks.

Provider popover

LLM details live in nav.

Guardrails and handoff

SAFETY

Know what can change

The server keeps writes narrow. Browser actions request specific mutations instead of giving the UI broad filesystem access.

Allowed targets

Slide specs

Structured JSON in the active presentation.

Presentation state

Context, variants, materials, and preferences.

Materials

Image files for the active deck.

Change discipline

○ Allowlisted paths

Unexpected targets are rejected.

○ Named actions

Add, remove, attach, compare, apply.

○ Separate archive

Sharing output is deliberate.

VALIDATION

Run checks before sharing

Validation stays subtle in the UI, but it is the final safety net before a PDF becomes the shared version.

— Studio, server, and scripts compile.

— Rendered slide bounds, text, and contrast.

OUTPUT

Archive the version you mean to share

Build creates current output. Archive only the PDF you mean to publish.

Artifact path

1

PDF

slides/output/<id>.pdf.

2

Pages

output/<id>/rendered-
pages.

3

Baseline

studio/baseline/<id>.

Routine

Rebuild

Run after slide or theme changes.

Gate

Run quality checks.

Archive

Update the checked-in copy.

How to read the project

If you are new to the codebase, start with the folder that matches the change.

Where to look

presentations/

Deck source, context, materials, state.

studio/

Browser app, server, renderer, baselines.

scripts/

Build, validation, archive commands.

Working habits

Small changes

Keep edits close to the boundary.

Docs follow code

Update guidance when workflows move.

Gate locally

Run quality checks before committing.

Current architecture bets

The useful path is still structured authoring, with local, desktop, and hosted surfaces sharing the same core.

Implemented baseline

Agent APIs

Expose versioned actions.

Desktop shell

Wrap the local studio.

Cloud path

Keep hosted storage aligned.

Boundaries

○ No full WYSIWYG chase

Structured decks are the point.

○ No second renderer

The DOM path remains authoritative.

○ Typed boundaries

Strict TS and fixtures catch drift.

Build new decks in stages

CREATION FLOW

Creation is staged

New decks start with a brief, move through an editable outline, then draft validated slides progressively.

— Capture intent first.

— Review the outline before slides.

BRIEF STAGE

Brief captures just enough

The first stage asks for the deck intent, not every styling and sourcing choice at once.

Required intent

Title

Name the deck early.

Audience

Say who needs it.

Objective

Define the job.

Keep out of the way

Optional details

Sources and style can open only when needed.

Compact help

Examples live behind small help affordances.

One action

The main button generates an outline.

OUTLINE STAGE

Approve structure before slides

The outline is the first generated artifact. Users can correct the story before full slide specs exist.

Review surface

1

Thesis

What the deck argues.

2

Narrative arc

How the story moves.

3

Slide titles

Every beat is editable.

Approval rules

Approve first

Slide drafting starts only from an accepted outline.

Stale if brief changes

Changed inputs clear approval.

Locked snapshot

Drafting uses the approved outline.

OUTLINE LOCKS

Regenerate only weak beats

A good outline rarely needs a full reset. The flow should let users keep strong beats and repair the rest.

Refinement tools

Lock

Preserve a strong outline slide.

Regenerate unlocked

Replan around saved beats.

Regenerate one

Replace only a weak slide.

What stays stable



Position

Locked slides keep their slot.



Meaning

Locked wording is preserved.



No stale draft

Creation waits for a current approval.

Draft one slide at a time

**After approval, slideotter materializes
slide specs sequentially and keeps partial
progress recoverable.**

— Use the locked outline snapshot.

— Show the active slide.

Theme after content

Real slides

Theme choices can start from a sentence or site URL, then get judged against real slides.

- Extract site colors as a theme seed.
- Tune palette and font.

GROUNDING

Sources join the outline

Sources and materials should enrich the structure stage, especially when evidence belongs to a specific beat.

Inputs to add

Source text

Paste notes or excerpts.

Slide notes

Attach evidence to one beat.

Images

Add visual material before drafting.

Grounding cues

Visible use

Show when sources shaped generation.

Compact refs

Use numbered references by default.

Regenerate path

Add material, then rebuild the outline.

Make the staged flow real

RUNTIME STATE

Draft state survives restarts

Creation should feel safe to pause. Draft choices stay in ignored runtime state until a deck is created.

Stored draft

1

Brief

Intent survives reloads.

2

Outline

Structure stays reviewable.

3

Theme

Choices remain available.

Write boundary

No git noise

Drafts stay ignored.

Deck later

Files appear on create.

Defaults

Missing fields normalize.

COVERAGE

Validate the staged path

Browser flow

Browser coverage now exercises creation, materials, length scaling, duplicate, and delete through the real UI.

- Upload and attach media.
- Scale and restore slides.

MAINTENANCE FOCUS

Keep the workbench sturdy

The main surfaces exist now. Next work should come from real demo decks and concrete maintenance friction.

Deepen next

User layouts

Extend from real needs.

Source misses

Improve retrieval from failures.

Hosted paths

Harden from deployments.

Keep intact

☐ Locked outline

Draft from the approved snapshot.

☐ Explicit apply

Candidates wait for approval.

☐ Docs move too

Roadmap, ADRs, and slides stay aligned.

TAKEAWAY

The basic habit

Keep the story, sources, materials, theme, preview, review, validation, and archive in one guarded loop.

- **Start with intent**

Choose a deck, write the brief, add sources, then edit the slides that serve it.

- **Review every option**

Use variants, outline plans, layouts, and scaling as proposals.

- **Ship only checked output**

Build, validate, refresh baselines when needed, and update the archive.

DAILY COMMANDS

Build

```
npm run build
```

Quality gate

```
npm run quality:gate
```